



Exploring Kernel Performance

Practical Notes

Introduction

This set of exercises is designed to help you be familiar with the HPC systems we will use, and to let you explore the performance of some of the key computational routines or functions used within deep neural networks (DNNs).

We will work on the EIDF for these exercises, and we will primarily be using the course VM for this initial practical in the course. Future practical sessions will focus more on GPU usage and exploring the machine learning frameworks commonly used for DNN construction, training, and inference. These will require using Kubernetes to run jobs on the GPU node, but we will provide details on how to do this in future practicals.

If you do not already have access to the EIDF, particularly the **eidf018** project, or do not know how to get on to the system, please **let a helper or lecturer know** and they will guide you through the process.

Getting started

1. Log on to EIDF virtual machine (VM) for this course using your student account. The VM IP address for this course is 10.24.3.9. You need to access it through a jump host, using the following command:

```
ssh -J USERNAME@eidf-gateway.epcc.ed.ac.uk  
USERNAME@10.24.3.9
```

You will need to replace `USERNAME` with your actual username for the `eidf018` project.

2. The source code for this practical is on Learn, but it is also in `/home/eidf018/eidf018/shared/mlas/`. Copy the file `kernels.tar` from this shared location into your directory.
3. Unpack it with the command `tar xvf kernels.tar`

If you do not know how to do any of the steps above, or are struggling to complete them, please let a demonstrator know so they can help.

Exercise 1: Matrix multiplication

Matrix multiplication is one of the key components of DNNs. Used for the weights and edges calculations on the forward propagation step of training and inference, and in some of the back propagation functionality as well, it is heavily used by DNNs. In this exercise, we will try running various implementations of matrix-matrix multiplications on CPUs to investigate their performance.

The code for this exercise is in the `Practical1` directory, in a file called `matmult.py`. This provides a number of different implementations of matrix-matrix multiplication, using both the `numpy` and `scipy` libraries, and for a range of data configurations.

The program has two parameters you can use to control how it runs:

```
repeats = 10
array_size = 128
```

`repeats` defines how many times each benchmark is repeated (we generally need to repeat the benchmarks a few times to get reliable times), and `array_size` defines the size of the matrix used (i.e. the matrix will be of size `array_size x array_size`).

Normally we would run on dedicated resources for your practical exercises, or for your coursework. However, for this exercise you can try just running locally in the login VM for this course, i.e.:

```
python3 matmult.py
```

This should run the code with the initial setup and print out timings for the various matrix-multiplication approaches implemented.

Now, look through the code in `matmult.py` and see if you can see what is happening, and why the performance may be different between the different versions.

Alter the code and run for the following configurations (you will need to manually change the values and run the code each time):

repeats	array_size	runtime
100	128	
1000	30	
1000	32	
100	64	
100	120	
100	136	
100	256	
10	1000	
10	1024	

What performance variations/trends do you see? These matrix multiplications are the core of DNN functionality, although they will be much fast on GPUs than they are on the CPUs we are using here.

Exercise 2: Activation functions

In this exercise we are looking at the activation functions that are applied to the output of the matrix multiplications of the weights with the edge input values. We have seen in the lectures there are a range of different activation functions.

Some of these have been implemented for you in `activation.py`. Run this code with the following command:

```
python3 activation.py
```

What performance differences can you see with the current activation functions?

There are a number of extra activation functions still to be implemented. Modify the code and add implementations for the following functions:

1. Sigmoid
2. TanH
3. Softmax

The definitions for these functions are included in comments in the code. You will also need to uncomment the places where these functions are called in the main code.

Once you have implemented the above activation functions, run the program again to see the performance these can achieve. Try running the activation functions on smaller or bigger arrays by modifying the `size` parameter, i.e. changing:

```
size = 1000000
```

To other values and re-running the program. Does performance change significantly?

Exercise 3: Putting it all together

Now you have Python versions of the matrix-multiplication and activation functions, you can put both together into a single program that first multiplies weights and edge data, and then applies an activation function to that data.

Modify `combined.py` to add in a matrix multiplication function followed by an activation function, remembering that the activation function will be applied to each row of the matrix. You will see in the code we have added an accumulate function between the matrix operation and the activation function. This is the sum of the weights * edges for each neuron in the layer, and is what is needed for the input to the activation function

Experiment with different sizes of matrices and with different activation functions. What part takes the most runtime, and how does this vary when you change sizes and/or activation functions?

Extra Exercises

For the `tanh` activation function, `numpy` provides an inbuilt implementation. Experiment with replacing your `tanh` implementation with `numpy.tanh(data)` to see if that makes a difference in the runtime. There is also a `scipy.softmax` function you can try replacing your `softmax` implementation with.

We have also provided you with C versions of the activation functions. You can compile these as follows:

```
gcc -O3 -o activation activation.c -lm
```

You can run the program with this command:

```
./activation
```

Try running with the same configuration as the python versions. What performance do you observe? Is it faster or slower than the python implementations?